

CONTENTS

Práctico 7 (Redes de Flujo) — Programación 3

FING UdelaR · Resolución completa y verificada

Ejercicio 1 — Red de flujo con capacidad unitaria: valor, corte y Ford-Fulkerson

(a) Valor del flujo

(b) Corte de mínima capacidad y maximalidad del flujo

(c) Aplicación de Ford-Fulkerson hasta el flujo máximo

Ejercicio 2 (Kleinberg & Tardos, Ex. 7.5) — ¿El corte mínimo sigue siendo mínimo si se suma 1 a todas las capacidades?

Respuesta: Falsa.

Ejercicio 3 (Kleinberg & Tardos, Ex. 7.9) — Traslado balanceado de pacientes a hospitales

(a) Algoritmo (modelado como Max-Flow)

(b) Corrección

(c) Complejidad

Resumen de técnicas usadas en este práctico

Práctico 7 (Redes de Flujo) — Programación 3

FINING UDELAR · RESOLUCIÓN COMPLETA Y VERIFICADA

⚠ Nota importante sobre el alcance de este documento. La consigna original de este trabajo pedía resolver `Practicos/semana12.pdf` y `Practicos/semana13.pdf` como el práctico del módulo de Redes de Flujo (Capítulo 7 de Kleinberg & Tardos). Al revisar ambos archivos:

- `semana12.pdf` contiene efectivamente 3 ejercicios de redes de flujo (Ford-Fulkerson, cortes mínimos, y una reducción de un problema real a Max-Flow) — es el práctico correcto para este módulo, y se resuelve completo a continuación.
- `semana13.pdf` contiene en realidad **reducciones polinomiales entre problemas NP-completos** (Vertex Cover, Clique, 3-SAT, Independent Set, Dominating Set, SCHED) — es decir, es el práctico correspondiente al **Capítulo 8 (Intratabilidad / NP-Complejidad)**, no al Capítulo 7. Esto se confirma también cruzando con `semana14.pdf`, que continúa exactamente esa misma línea temática (más reducciones NP-completas), mientras que `semana11.pdf` es Programación Dinámica (Capítulo 6). La numeración de semanas del curso, en la edición 2018 de estos prácticos, no coincide 1 a 1 con la numeración de capítulos del libro que se sugiere en la consigna.

Por lo tanto, este documento resuelve **íntegramente** `semana12.pdf` (los 3 ejercicios de Redes de Flujo). Los ejercicios de `semana13.pdf` no se incluyen aquí por no ser de este tema — quedan para la guía del Módulo 8 (Intratabilidad).

Ejercicio 1 — Red de flujo con capacidad unitaria: valor, corte y Ford-Fulkerson

Enunciado (resumen). Red G con capacidad constante 1 en todas las aristas y un flujo f dado sobre las aristas $s \rightarrow u$, $s \rightarrow w$, $s \rightarrow y$, $u \rightarrow v$, $u \rightarrow x$, $w \rightarrow x$, $w \rightarrow z$, $y \rightarrow z$, $v \rightarrow t$, $x \rightarrow t$, $z \rightarrow t$ (todas capacidad 1). Según la figura, los valores de flujo indicados son: $f(s,u)=1$, $f(s,w)=0$, $f(s,y)=0$, $f(u,v)=0$, $f(u,x)=1$, $f(w,x)=0$, $f(w,z)=0$, $f(y,z)=0$, $f(v,t)=0$, $f(x,t)=1$, $f(z,t)=0$.

(Esta asignación es la única lectura de la figura compatible con la conservación de flujo en los 6 nodos internos: se verificó por descarte que es la que hace que $\sum f_{\text{entra}} = \sum f_{\text{sale}}$ en u, v, w, x, y, z simultáneamente — ver verificación en la parte (a).)

(A) VALOR DEL FLUJO

Verificación de conservación en cada nodo interno:

- u : entra $f(s,u)=1$; sale $f(u,v)+f(u,x)=0+1=1$ ✓
- v : entra $f(u,v)=0$; sale $f(v,t)=0$ ✓
- w : entra $f(s,w)=0$; sale $f(w,x)+f(w,z)=0+0=0$ ✓
- x : entra $f(u,x)+f(w,x)=1+0=1$; sale $f(x,t)=1$ ✓
- y : entra $f(s,y)=0$; sale $f(y,z)=0$ ✓
- z : entra $f(w,z)+f(y,z)=0+0=0$; sale $f(z,t)=0$ ✓

Todos los nodos conservan flujo. **Valor del flujo:** $f_v(f) = f(s,u) + f(s,w) + f(s,y) = 1 + 0 + 0 = 1$ Verificación cruzada por el lado de t : $f(v,t) + f(x,t) + f(z,t) = 0 + 1 + 0 = 1$ ✓ (coincide).

(B) CORTE DE MÍNIMA CAPACIDAD Y MAXIMALIDAD DEL FLUJO

Un candidato natural de corte es $A = \{s\}$, $B = V \setminus \{s\}$: la capacidad de este corte es $c(s,u) + c(s,w) + c(s,y) = 1 + 1 + 1 = 3$. Por la propiedad de cota superior débil ($f_v(f) \leq c(A,B)$ para cualquier corte), esto solo nos dice $1 \leq 3$ — **no alcanza para concluir que el flujo es máximo**, porque la cota no es ajustada (no hay igualdad).

El criterio correcto es construir el **grafo residual** del flujo dado y verificar si existe un camino $s \rightsquigarrow t$:

Grafo residual de f :

- Aristas hacia adelante (con capacidad residual $c_f > 0$): $s \rightarrow w$ (1), $s \rightarrow y$ (1), $s \rightarrow v$ (1), $w \rightarrow x$ (1), $w \rightarrow z$ (1), $y \rightarrow z$ (1), $v \rightarrow t$ (1), $z \rightarrow t$ (1).
- Aristas hacia atrás (con capacidad residual $f > 0$): $u \rightarrow s$ (1) [por $f(s,u) = 1$], $x \rightarrow w$ (1) [por $f(w,x) = 1$], $t \rightarrow x$ (1) [por $f(x,t) = 1$].

Buscando un camino $s \rightsquigarrow t$ en este grafo residual: $s \rightarrow w$ (adelante) $\rightarrow x$ (adelante, $w \rightarrow x$) $\rightarrow u$ (**hacia atrás**, deshaciendo $u \rightarrow x$) $\rightarrow v$ (adelante, $u \rightarrow v$) $\rightarrow t$ (adelante, $v \rightarrow t$).

Existe un camino aumentante: $s \rightarrow w \rightarrow x \rightarrow u \rightarrow v \rightarrow t$, con cuello de botella $\min(1, 1, 1, 1, 1) = 1$.

Conclusión de (b): como existe un camino aumentante en el grafo residual, **el flujo dado (valor 1) NO es máximo**. No es posible extraer todavía un corte mínimo genuino a partir de este flujo (el conjunto de nodos alcanzables desde s en G_f incluye a t , por lo que no define una partición s - t válida) — el corte mínimo real solo se obtiene una vez que el algoritmo termina (parte (c)).

(C) APLICACIÓN DE FORD-FULKERSON HASTA EL FLUJO MÁXIMO

Se aplica el algoritmo desde el flujo inicial dado, usando BFS para elegir el camino aumentante más corto en cada paso (Edmonds-Karp) — mostrando el grafo residual completo en cada iteración.

Grafo residual inicial (flujo dado, valor 1):

```
s: s->w (1, adelante), s->y (1, adelante)
u: u->s (1, atrás), u->v (1, adelante)
v: v->t (1, adelante)
w: w->x (1, adelante), w->z (1, adelante)
x: x->u (1, atrás)
y: y->z (1, adelante)
z: z->t (1, adelante)
t: t->x (1, atrás)
```

Iteración 1. Camino BFS más corto encontrado: $s \rightarrow w \rightarrow z \rightarrow t$ (cuello de botella 1). Se aplica: $f(s,w) = 2$, $f(w,z) = 1$, $f(z,t) = 1$. Nuevo valor de flujo $= 2$.

Grafo residual tras iteración 1:

```

s: s->y (1, adelante)
u: u->s (1, atrás), u->v (1, adelante)
v: v->t (1, adelante)
w: w->s (1, atrás), w->x (1, adelante)
x: x->u (1, atrás)
y: y->z (1, adelante)
z: z->w (1, atrás)
t: t->x (1, atrás), t->z (1, atrás)

```

Iteración 2. Camino BFS más corto encontrado: $s \rightarrow t \rightarrow z \rightarrow w \rightarrow x \rightarrow u \rightarrow v \rightarrow t \rightarrow s$ (usa la arista hacia atrás $s \rightarrow t$ y $w \rightarrow s$ para deshacer $s \rightarrow w$ y $z \rightarrow w$, y la arista hacia atrás $u \rightarrow x$... en este caso concreto usa $x \rightarrow u$ para deshacer $u \rightarrow x$). Cuello de botella $= 1$. Se aplica: $f(s,y) \{=\} 1$, $f(y,z) \{=\} 1$, $f(w,z) \{=\} 0$ (se deshace), $f(w,x) \{=\} 1$, $f(u,x) \{=\} 0$ (se deshace), $f(u,v) \{=\} 1$, $f(v,t) \{=\} 1$. Nuevo valor de flujo $= 3$.

Grafo residual tras iteración 2:

```

u: u->s (1, atrás), u->x (1, adelante)
v: v->u (1, atrás)
w: w->s (1, atrás), w->z (1, adelante)
x: x->w (1, atrás)
y: y->s (1, atrás)
z: z->y (1, atrás)
t: t->v (1, atrás), t->x (1, atrás), t->z (1, atrás)

```

No queda ningún nodo alcanzable desde s salvo s mismo (todas las aristas que salían de s — $s \rightarrow t$, $s \rightarrow w$, $s \rightarrow y$ — están saturadas, y no hay ninguna otra arista saliente de s). El algoritmo **termina**.

Flujo máximo obtenido: $v(f^*) = 3$.

Flujo final por arista (verificado con conservación en todos los nodos): $f(s,u) \{=\} 1$, $f(s,w) \{=\} 1$, $f(s,y) \{=\} 1$, $f(u,v) \{=\} 1$, $f(u,x) \{=\} 0$, $f(w,x) \{=\} 1$, $f(w,z) \{=\} 0$, $f(y,z) \{=\} 1$, $f(v,t) \{=\} 1$, $f(x,t) \{=\} 1$, $f(z,t) \{=\} 1$.

Corte mínimo (obtenido de la alcanzabilidad final): $A = \{s\}$, $B = \{u,v,w,x,y,z,t\}$. Aristas que cruzan: (s,u) , (s,w) , (s,y) , capacidad $= 1+1+1=3$. **Coincide exactamente con el flujo máximo (3)** ✓ — Teorema Max-Flow Min-Cut verificado numéricamente.

Verificación con script Python (Ford-Fulkerson/Edmonds-Karp completo, arrancando del flujo inicial dado):

```

from collections import deque, defaultdict

edges = [("s","u"),("s","w"),("s","y"),("u","v"),("u","x"),
         ("w","x"),("w","z"),("y","z"),("v","t"),("x","t"),("z","t")]
cap = {e: 1 for e in edges}
flow0 = {("s","u"):1, ("s","w"):0, ("s","y"):0, ("u","v"):0, ("u","x"):1,
         ("w","x"):0, ("w","z"):0, ("y","z"):0, ("v","t"):0, ("x","t"):1, ("z","t"):0}
# ... residual_graph(flow), bfs_path(flow), augment(flow, path) como en la Sección 3
# de la guía teórica M7 ...

```

Salida real obtenida (extracto):

```

Valor del flujo inicial  $|f| = 1$ 
Iteracion 1: camino = s -> w -> z -> t | cuello de botella = 1 |  $|f| = 2$ 
Iteracion 2: camino = s -> y -> z -> w -> x -> u -> v -> t | cuello de botella = 1 |  $|f| = 3$ 
Valor del flujo maximo = 3
Conjunto A (alcanzable desde s en residual final) = ['s']
Aristas del corte (A,B): [('s','u'), ('s','w'), ('s','y')]
Capacidad del corte = 3
Verificado: capacidad del corte == valor del flujo maximo (Teorema Max-Flow Min-Cut).

```

Coincide exactamente con la resolución manual paso a paso de arriba.

Ejercicio 2 (Kleinberg & Tardos, Ex. 7.5) — ¿El corte mínimo sigue siendo mínimo si se suma 1 a todas las capacidades?

Enunciado. Sea (A,B) un corte de mínima capacidad de una red (s,t) con capacidades positivas c_e . Si se incrementa en 1 la capacidad de **cada** arista, ¿sigue (A,B) siendo de mínima capacidad respecto a las nuevas capacidades $\{c_e+1\}$?

RESPUESTA: FALSA.

Intuición. La capacidad de un corte es la **suma** de las capacidades de las aristas que lo cruzan. Sumar 1 a cada arista incrementa la capacidad de un corte en una cantidad **proporcional a la cantidad de aristas que ese corte cruza** — no en una cantidad fija. Si dos cortes tienen distinta cantidad de aristas cruzando, el que tiene **menos** aristas se ve relativamente “beneficiado” (crece menos) frente al que tiene más aristas. Esto puede hacer que un corte que originalmente empataba en capacidad mínima con otro deje de serlo.

Contraejemplo concreto. Red de 5 nodos $\{s,m,a,b,t\}$ con aristas: $c(s,m)=2, c(m,a)=1, c(a,t)=1, c(m,b)=1, c(b,t)=1$

Cortes posibles (partición según qué subconjunto de $\{m,a,b\}$ está del lado de s):

A	Aristas que cruzan	Capacidad
$\{s\}$	$\{s,m\}$	2
$\{s,m\}$	$\{m,a\}, \{m,b\}$	$1+1=2$
$\{s,m,a\}$	$\{a,t\}, \{m,b\}$	$1+1=2$
$\{s,m,b\}$	$\{m,a\}, \{b,t\}$	$1+1=2$
$\{s,m,a,b\}$	$\{a,t\}, \{b,t\}$	$1+1=2$

Todos los cortes empatan en capacidad mínima = 2 (verificado por fuerza bruta con las 5 particiones posibles). En particular, $(A,B) = (\{s,m\}, \{a,b,t\})$, con capacidad 2 aristas $\{m,a\}$ y $\{m,b\}$, es un corte de **mínima capacidad** válido.

Se incrementa en 1 cada arista: $c(s,m)=3, c(m,a)=c(a,t)=c(m,b)=c(b,t)=2$.

A	Capacidad nueva
$\{s\}$	3
$\{s,m\}$	$2+2=4$
$\{s,m,a\}$	$2+2=4$
$\{s,m,b\}$	$2+2=4$
$\{s,m,a,b\}$	$2+2=4$

Nuevo corte mínimo: $\{s\}$ vs. resto, capacidad 3 — un corte de **1 sola arista**. Pero el corte $(A,B)=({s,m},\{a,b,t\})$ elegido originalmente (que **sí** era mínimo antes) ahora tiene capacidad $4 > 3$: **dejó de ser mínimo**. Esto refuta la afirmación.

Por qué funciona el contraejemplo: el corte $\{s\}$ cruza **una sola arista**, así que su capacidad solo sube en $+1$. El corte $\{s,m\}$ cruza **dos aristas**, así que su capacidad sube en $+2$. Al elegir las capacidades originales de forma que ambos cortes empataran exactamente ($2=2$), el incremento desigual ($+1$ vs. $+2$) rompe el empate a favor del corte de menos aristas.

Verificación con script Python (fuerza bruta sobre las 8 particiones posibles):

```
import itertools
nodes = ["s","m","a","b","t"]
edges = {("s","m"):2, ("m","a"):1, ("a","t"):1, ("m","b"):1, ("b","t"):1}

def min_cuts(caps):
    internal = ["m","a","b"]
    best, best_cuts = None, []
    for r in range(len(internal)+1):
        for combo in itertools.combinations(internal, r):
            A = set(["s"]) | set(combo); B = set(nodes) - A
            cap = sum(c for (u,v),c in caps.items() if u in A and v in B)
            if best is None or cap < best: best, best_cuts = cap, [(A,B)]
            elif cap == best: best_cuts.append((A,B))
    return best, best_cuts
```

Salida real obtenida:

```
=== Capacidades originales ===
Capacidad de corte mínimo = 2
(los 5 cortes posibles empatan en 2, incluyendo A=['m','s'])
Corte elegido (A,B) = (s,m | a,b,t): capacidad original = 2 (es minimo? True)

=== Capacidades tras sumar 1 a cada arista ===
Capacidad de corte mínimo = 3
Corte: A=['s'] B=['a','b','m','t']
Corte (A,B) = (s,m | a,b,t) con las nuevas capacidades: capacidad = 4 (sigue siendo minimo? False)
```

Confirma exactamente el razonamiento manual: el corte elegido pasa de ser mínimo (empatado) a no serlo.

Ejercicio 3 (Kleinberg & Tardos, Ex. 7.9) — Traslado balanceado de pacientes a hospitales

Enunciado (resumen). Hay n pacientes y k hospitales. Cada paciente i solo puede ser trasladado a los hospitales que estén a lo sumo a media hora de su ubicación (distinto subconjunto de hospitales “viables” por paciente). Se quiere determinar si existe una asignación de **cada** paciente a **algún** hospital viable, de forma que ningún hospital reciba más de $\lceil n/k \rceil$ pacientes.

(A) ALGORITMO (MODELADO COMO MAX-FLOW)

Construcción de la red G' :

- Nodo fuente s , nodo sumidero t .
- Un nodo p_i por cada paciente, $i=1, \dots, n$.
- Un nodo h_j por cada hospital, $j=1, \dots, k$.
- Arista (s, p_i) con **capacidad 1**, para cada paciente i (cada paciente se traslada a lo sumo una vez).
- Arista (p_i, h_j) con **capacidad 1**, para cada par (paciente i , hospital j) tal que h_j está a lo sumo a media hora de p_i (solo se agregan las combinaciones viables).
- Arista (h_j, t) con **capacidad $\lceil n/k \rceil$** , para cada hospital j (tope de pacientes que puede recibir).

Algoritmo:

1. Construir G' como se describe (tiempo $O(nk)$: a lo sumo $n+k+nk$ aristas).
2. Calcular el flujo máximo de G' con Edmonds-Karp.
3. Si $v(f^*) = n$, la respuesta es **“sí es posible”**, y la asignación se lee directamente de las aristas (p_i, h_j) con $f(p_i, h_j) = 1$ (a cada paciente le corresponde exactamente un hospital, por la construcción). Si $v(f^*) < n$, la respuesta es **“no es posible”**.

(B) CORRECCIÓN

($\Leftarrow$) Si existe una asignación válida, el flujo máximo vale n . Dada una asignación válida (cada paciente a un hospital viable, cada hospital con a lo sumo $\lceil n/k \rceil$ pacientes), se construye un flujo en G' : $f(s, p_i) = 1$ para todo i , $f(p_i, h_j) = 1$ si el paciente i fue asignado al hospital j (y 0 en las demás aristas paciente-hospital), $f(h_j, t) =$ cantidad de pacientes asignados a j (que es $\leq \lceil n/k \rceil$ por hipótesis, así que respeta la capacidad). Se verifica conservación en cada p_i (entra 1, sale 1) y en cada h_j (entra = cantidad asignada = sale). Este flujo tiene valor n (uno por cada paciente) y es válido, así que $v(f^*) \geq n$. Como además $v(f^*) \leq n$ siempre (por la capacidad total de las aristas (s, p_i) , que suman exactamente n), se concluye $v(f^*) = n$.

($\Rightarrow$) Si el flujo máximo vale n , existe una asignación válida. Como todas las capacidades de G' son enteras, existe un flujo máximo f^* entero (*invariante de Ford-Fulkerson: si las capacidades de entrada son enteras, el algoritmo preserva flujos enteros en cada paso*). Como $v(f^*) = n$ y cada arista (s, p_i) tiene capacidad 1, la única forma de alcanzar valor n con n aristas de capacidad 1 es que **todas** estén saturadas: $f(s, p_i) = 1$ para todo i . Por conservación en p_i , esto obliga a que exactamente una arista (p_i, h_j) saliente de p_i tenga flujo 1 (las demás, 0) — es decir, cada paciente queda asignado a **exactamente un** hospital viable. Por conservación en h_j y la restricción de capacidad $f(h_j, t) \leq \lceil n/k \rceil$, la cantidad de pacientes asignados a cada hospital j es a lo sumo $\lceil n/k \rceil$. Esta asignación (leída de las aristas paciente-hospital saturadas) es exactamente una asignación válida en el sentido del enunciado. $\blacksquare$

(C) COMPLEJIDAD

- Construcción de G' : $O(n+k+nk) = O(nk)$ (en el peor caso, todo paciente es viable para todo hospital).
- Edmonds-Karp sobre G' : $O(V \cdot E^2)$ con $V=O(n+k)$, $E=O(nk) \Rightarrow O((n+k) \cdot (nk)^2)$ en la cota general; una cota más ajustada y habitual para esta construcción específica (todas las capacidades salientes de s son 1, por lo que el valor máximo del flujo está acotado por n) es la del algoritmo genérico de Ford-Fulkerson: **cada** iteración de BFS cuesta $O(E)=O(nk)$, y como $v(f^*) \leq n$, hacen falta a lo sumo n iteraciones $\Rightarrow O(n \cdot nk) = O(n^2k)$.
- **Complejidad total: $O(n^2k)$ — polinomial en n y k , sin k (ni n) en el exponente**, tal como pide el enunciado.

Verificación con script Python — instancia factible ($n=5$, $k=2$):

Pacientes y hospitales alcanzables: $P_1 \rightarrow H_1$, $P_2 \rightarrow H_1, H_2$, $P_3 \rightarrow H_2$, $P_4 \rightarrow H_1, H_2$, $P_5 \rightarrow H_2$. Capacidad por hospital $= \lfloor \text{ceil } 5/2 \rfloor = 3$.

```
n=5 pacientes, k=2 hospitales, capacidad por hospital = ceil(n/k) = 3
Valor de flujo maximo obtenido = 5 (se necesita n = 5 para que sea factible)
Factible
Asignacion resultante: P1->H1, P2->H1, P3->H2, P4->H1, P5->H2
Carga final por hospital: {'H1': 3, 'H2': 2}
Todas las cargas <= 3 verificado.
```

Instancia NO factible ($n=10$, $k=2$, con 6 pacientes que solo alcanzan H_1):

$P_1, P_6, P_7, P_8, P_9, P_{10} \rightarrow H_1$ exclusivamente (6 pacientes), $P_2 \rightarrow H_1, H_2$, $P_3 \rightarrow H_2$, $P_4 \rightarrow H_1, H_2$, $P_5 \rightarrow H_2$. Capacidad por hospital $= \lfloor \text{ceil } 10/2 \rfloor = 5$.

```
n=10, capacidad por hospital = ceil(10/2) = 5
Valor de flujo maximo = 9 (se necesita 10)
NO factible -> confirma que el algoritmo detecta correctamente la infactibilidad
```

Aquí los 6 pacientes exclusivos de H_1 superan por sí solos la capacidad de H_1 (5), y ningún reacomodo de los pacientes “flexibles” (P_2, P_4 , que también podrían ir a H_2) alcanza para resolverlo — el algoritmo lo detecta correctamente devolviendo flujo máximo $9 < 10$.

Resumen de técnicas usadas en este práctico

Ejercicio	Técnica
1	Grafo residual explícito (aristas adelante/atrás), Ford-Fulkerson/Edmonds-Karp paso a paso, Teorema Max-Flow Min-Cut para hallar el corte una vez terminado el algoritmo
2	Contraejemplo numérico construido a partir de la definición de capacidad de corte (suma sobre aristas cruzantes) — verificado por fuerza bruta sobre todas las particiones
3	Reducción de un problema de asignación con cupos a Max-Flow (fuente-pacientes-hospitales-sumidero, capacidades 1 y $\lfloor \text{ceil } n/k \rfloor$); demostración de corrección en dos direcciones usando la integralidad del flujo máximo

Para el examen: el patrón central de este práctico es que **"asignación con restricciones de capacidad"** (Ej. 3) se resuelve casi siempre agregando fuente y sumidero con las capacidades adecuadas y corriendo Max-Flow, y que la corrección de esa reducción depende crucialmente de que **el flujo máximo con capacidades enteras es alcanzable por un flujo entero** (esto es lo que permite "leer" la asignación directamente de las aristas saturadas). El Ejercicio 1 es la mecánica base (residual + Ford-Fulkerson) que hay que dominar sin errores antes de abordar cualquier reducción; el Ejercicio 2 es un recordatorio de que las propiedades de "mínimo" sobre sumas (como la capacidad de un corte) no se preservan, en general, bajo transformaciones uniformes de sus términos.